

Software Engineering Cheat Sheet

Software Development Life Cycles (SDLC)

- Waterfall: Sequential phases; used in aerospace and defense (e.g., NASA projects).
- Agile: Iterative sprints; popular at Spotify and Atlassian for rapid delivery.
- DevOps: Continuous integration/deployment; Netflix uses to deploy hundreds of times/day.

DRY (Don't Repeat Yourself)

- Principle: Eliminate duplication.
- Use Case: Centralized utility libraries at Netflix reduce code duplication across microservices.

ACID (Atomicity, Consistency, Isolation, Durability)

- Principle: Reliable DB transactions.
- Use Case: Banking systems (e.g., JPMorgan Chase) enforce ACID for transaction integrity.

SOLID Principles

- Single Responsibility: Class has one reason to change (e.g., Logger class).
 - Open/Closed: Modules open for extension, closed for modification (e.g., plugin systems).
 - Liskov Substitution: Subtypes must replace base types without altering behavior.
 - Interface Segregation: Many client-specific interfaces instead of one general.
 - Dependency Inversion: High-level modules shouldn't depend on low-level modules.
- Use Case: SOLID guides architecture at Amazon to maintain scalable codebases.*

KISS (Keep It Simple, Stupid)

- Principle: Simplify designs; avoid over-engineering.
- Use Case: Slack started with a monolithic codebase before splitting, focusing on simplicity.

Modularity

- Principle: Break system into independent modules.
- Use Case: Linux kernel modules allow loading/unloading features without rebooting.

YAGNI (You Aren't Gonna Need It)

- Principle: Don't add functionality until necessary.
- Use Case: Startups like Basecamp ship minimal features to validate before building more.

Design Patterns

- Singleton: One instance (e.g., logging).
 - Factory: Create objects without specifying class (e.g., UI component factories).
 - Observer: Publish/subscribe (e.g., event handling in JavaScript).
 - Strategy: Define interchangeable algorithms (e.g., sorting strategies in Java).
- Use Case: Design patterns in Spring Framework underpin scalable enterprise apps.*

Separation of Concerns

- Principle: Distinct features handled by separate sections.
- Use Case: MVC in Ruby on Rails separates data, UI, and control logic.